Software Requirements Specification

for

Peer-to-Peer University Tutoring Platform with Personalized Recommendation System

Prepared by

Yukta Menon	se23ucse192
Sujan Vadla	se23ucse178
Hridya Reddy	se23uari066
Pranishk Reddy	se23uari099
Bharat Reddy	se23uari020

Instructor:	**Avinash Chauhan**
Course:	**Software engineering**
Group number:	***43***
Teaching Assistant:	**Sai Srithaja**
Date:	**12/03/26**

1 Introduction

1.1 Document Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Peer-to-Peer University Tutoring Platform with Personalized Recommendation System. The document defines the system features, behavior, constraints, and interactions between users and the platform.

The purpose of this document is to provide a clear and detailed description of the requirements for developers, testers, project members, and instructors involved in the project. This SRS serves as a reference for the design, development, testing, and evaluation of the tutoring platform.

1.2 Product Scope

The Peer-to-Peer University Tutoring Platform is a web-based application designed to help students within the same university connect for academic tutoring. The platform allows students to register either as tutors or learners and manage tutoring sessions through an online interface.

Students can search for tutors based on subjects, availability, ratings, and price. Learners can request tutoring sessions, communicate with tutors through a booking-based messaging system, and leave reviews after sessions are completed. The platform also includes a recommendation system that suggests tutors and learning paths based on user activity and booking history.

The goal of the system is to improve access to tutoring resources within the university community while providing a convenient and structured way for students to collaborate academically.

1.3 Intended Audience and Document Overview

This document is intended for several audiences involved in the development and evaluation of the system.

Developers will use this document to understand system functionality and technical requirements during implementation. Testers will use the SRS to verify that the system meets the required specifications. Project team members will use it as a guideline for system design and development.

The instructor and course staff will use the document to evaluate the completeness and clarity of the system requirements.

The document is organized into multiple sections. Section 1 introduces the project and its objectives. Section 2 provides an overall description of the system, including system context and general functionality. Section 3 describes the specific functional requirements and system interactions. Section 4 outlines non-functional requirements such as performance, security, and software quality attributes. Section 5 describes additional system requirements and information.

1.4 Definitions, Acronyms and Abbreviations

API – Application Programming Interface
DB – Database
UI – User Interface
UX – User Experience
SRS – Software Requirements Specification
SOW – Statement of Work
P2P – Peer-to-Peer
CRUD – Create, Read, Update, Delete
SQL – Structured Query Language

1.5 Document Conventions

This document follows the IEEE Software Requirements Specification formatting guidelines. The text uses standard formatting with section headings and numbered subsections.

Functional requirements are labeled using identifiers such as F1, F2, and F3. Non-functional requirements are labeled using identifiers such as P1 for performance requirements. Tables are used where appropriate to organize system data and definitions.

The document uses clear and structured language to describe the system behavior and expected functionality.

1.6 References and Acknowledgments

Statement of Work – Peer-to-Peer University Tutoring Platform with Personalized Recommendation System.

React Documentation – https://react.dev

Node.js Documentation – https://nodejs.org

PostgreSQL Documentation – https://postgresql.org

IEEE Software Requirements Specification Template.

---

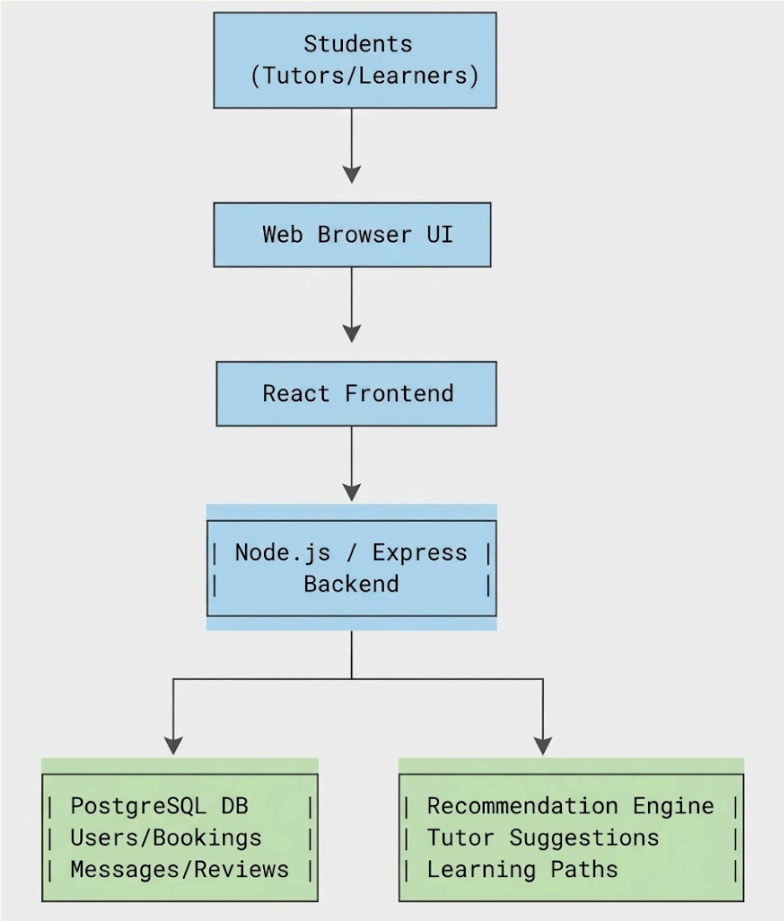
2 Overall Description

2.1 Product Overview

The Peer-to-Peer University Tutoring Platform is a web-based system designed to help students find and schedule tutoring sessions with other students within the same university. The platform allows students to act as either tutors or learners depending on their preferences and expertise.

The system consists of three major components: the frontend user interface, the backend application server, and the database system. The frontend interface allows users to interact with the system through a web browser. The backend server processes requests and manages application logic. The database stores user data, booking information, tutor profiles, and reviews.

The system also includes a recommendation module that analyzes booking history and subject relationships to suggest tutors and learning paths to students.



2.2 Product Functionality

The platform provides several key functionalities that support tutoring interactions between students.

- User registration and authentication
- Tutor profile creation and management
- Tutor availability scheduling
- Search functionality for tutors by subject, rating, price, and availability
- Session booking between learners and tutors
- Booking management with different booking states
- Messaging system for communication within a booking
- Review and rating system after session completion
- Tutor recommendation system based on booking history
- Learning path recommendation system based on subject relationships

2.3 Design and Implementation Constraints

Several design and implementation constraints apply to the development of the system.

The frontend of the system will be developed using React. The backend server will be implemented using Node.js and Express. PostgreSQL will be used as the database system to store user information, bookings, reviews, and messages.

The system will operate within a single university environment, meaning all users belong to the same institution. The system will not integrate a real payment gateway since tutoring sessions are assumed to be arranged without financial transactions through the platform.

The project must also follow software engineering design principles and UML modeling techniques learned during the course.

2.4 Assumptions and Dependencies

Several assumptions are made during the development of the system.

All users belong to the same university and will access the platform using university credentials. Students will manually mark tutoring sessions as completed once the session ends.

Tutors will specify their weekly recurring availability for tutoring sessions. Messaging will only be allowed between users who share an active booking.

The system depends on the availability of web browsers for user interaction, a backend server environment capable of running Node.js, and a PostgreSQL database server for storing application data.

---

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

Users will interact with the system through a web-based interface accessible through modern web browsers. The interface will be designed to provide an intuitive and user-friendly experience suitable for university students.

The main user interface components include the login and registration pages, tutor search interface, tutor profile pages, booking management pages, messaging interface, and review submission pages. The interface will allow users to navigate easily between these sections and perform tutoring-related activities.

3.1.2 Hardware Interfaces

The system does not require specialized hardware devices. Users can access the platform using standard computing devices such as laptops, desktops, tablets, or smartphones with internet connectivity.

The system will run on a server capable of hosting the backend application and database.

3.1.3 Software Interfaces

The system includes several internal software interfaces between different components.

The React frontend communicates with the backend server using REST API requests. The backend server processes these requests and retrieves or updates data stored in the PostgreSQL database.

The recommendation module interacts with booking history and user activity data to generate tutor and learning path recommendations.

3.2 Functional Requirements

F1: User Registration
The system shall allow students to create accounts using their university email addresses.

F2: User Authentication
The system shall allow registered users to log in securely using their credentials.

F3: Tutor Profile Management
The system shall allow tutors to create and update profiles that include subjects taught, hourly rate, description, and availability schedule.

F4: Tutor Search
The system shall allow learners to search for tutors using filters such as subject, price, rating, and availability.

F5: Tutor Profile Viewing
The system shall allow users to view detailed tutor profiles including ratings, reviews, and subject expertise.

F6: Session Booking
The system shall allow learners to request tutoring sessions from tutors.

F7: Booking Management
The system shall track booking states such as pending, accepted, completed, or cancelled.

F8: Messaging System
The system shall allow tutors and learners to communicate through messages associated with a specific booking.

F9: Review and Rating System
The system shall allow learners to submit ratings and reviews after a tutoring session has been completed.

F10: Tutor Recommendation System
The system shall recommend tutors based on a student's booking history and subject preferences.

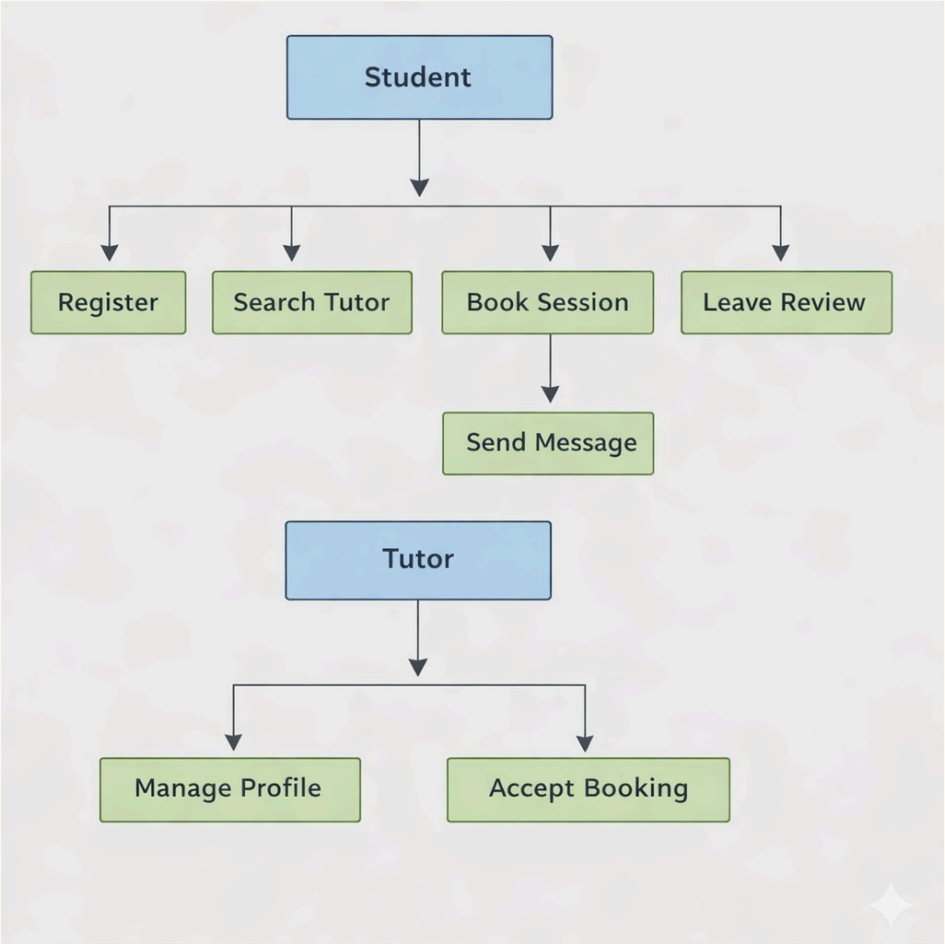
F11: Learning Path Recommendation
The system shall suggest learning paths to students based on subject relationships and previous tutoring sessions.

3.3 Use Case Model

The system includes several use cases that represent interactions between users and the platform.

Actors in the system include Students (learners), Tutors, and the System itself.

Primary use cases include registering for the platform, logging into the system, searching for tutors, booking tutoring sessions, messaging within bookings, submitting reviews, and receiving recommendations.



3.3.1 Use Case U1 – Search Tutor

Author – Bharat Reddy

Purpose – The purpose of this use case is to allow students to search for tutors who teach specific subjects.

Requirements Traceability – F4, F5

Priority – High

Preconditions – The user must be logged into the system.

Postconditions – The system displays a list of tutors matching the search criteria.

Actors – Student

Flow of Events

Basic Flow

1. The student enters a subject name or search criteria.
2. The system retrieves tutors matching the criteria.

3. The system displays the list of tutors.
4. The student selects a tutor profile to view more information.

Alternative Flow

If no tutors match the search criteria, the system displays a message indicating that no results were found.

Exceptions

System errors may occur if the database connection fails.

---

4 Other Non-functional Requirements

4.1 Performance Requirements

P1: The system shall display tutor search results within three seconds under normal usage conditions.

P2: The system shall support multiple simultaneous users accessing the platform without significant performance degradation.

P3: Messaging updates should appear within two seconds after a message is sent.

4.2 Safety and Security Requirements

User accounts must be protected using secure authentication mechanisms. Passwords must be stored using secure hashing techniques to prevent unauthorized access.

Messaging functionality must be restricted so that only users involved in a booking can view and send messages within that booking thread.

The system must protect user data such as personal information, reviews, and booking records from unauthorized access or modification.

4.3 Software Quality Attributes

Reliability

The system should operate reliably with minimal downtime and handle user requests consistently without system failures.

Usability

The platform should provide an intuitive and easy-to-use interface that allows students to quickly search for tutors, book sessions, and communicate with other users.

Maintainability

The system architecture should be modular to allow developers to update or modify components without affecting the entire system.

Scalability

The platform should support an increasing number of users and tutoring sessions without significant performance issues.

---

5 Other Requirements

The system database must store several types of data including user accounts, tutor profiles, subjects, bookings, reviews, and messages.

The system should maintain relationships between these entities to ensure proper functionality of booking management, messaging, and recommendation features.

Appendix A – Data Dictionary

Data Element	Description
user_id	Unique identifier for each user
tutor_id	Unique identifier for each tutor

booking_id	Unique identifier for each tutoring session
subject	Subject being taught in a tutoring session
rating	Rating given to a tutor after a session
message_id	Unique identifier for each message
review	Written feedback submitted by learners

Appendix B – Group Log

Meeting 1 – Initial project planning and discussion of project scope.

Meeting 2 – Requirements analysis and system design planning.

Meeting 3 – Database schema design and backend setup.

Meeting 4 – Implementation of tutor search and booking functionality.

Meeting 5 – Development of messaging system and review features.

Meeting 6 – Final testing, debugging, and documentation preparation.